B.TECH COMPUTER SCIENCE
THIRD YEAR PROJECT REPORT

Amazon Review Sentiment Analysis

Classifying Amazon Product Reviews as Positive or Negative using NLP & Machine Learning

Submitted by

Himanshu Gupta
B.Tech Computer Science | 3rd Year

GitHub Repository
https://github.com/guptahimansh0002/amazon_review_sentiment_analysis/blob/main/notebooks/sentiment_analysis_tfidf_logistic_regression_model.ipynb

Submission Date: 2026

Certificate

This is to certify that the project entitled Amazon Review Sentiment Analysis has been carried out by Himanshu Gupta, a student of B.Tech Computer Science (3rd Year), as a part of the curriculum requirement.

The project work is original and has been completed under proper academic supervision. The results presented in this report are genuine and have not been submitted elsewhere for any degree or examination.

Project Guide / Supervisor: _____

Department: Computer Science & Engineering

Date: _____

Declaration

I, Himanshu Gupta, hereby declare that the project entitled Amazon Review Sentiment Analysis submitted in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science is a record of original work done by me.

The project work has not been submitted for the award of any other degree or diploma. All information used from other sources has been duly acknowledged.

Name: Himanshu Gupta

Date: _____

Signature: _____

Acknowledgement

I would like to express my sincere gratitude to all those who helped me in completing this project successfully.

I am deeply thankful to my Project Guide and faculty members of the Computer Science Department for their invaluable guidance, encouragement, and support throughout the development of this project.

I am also grateful to my institution for providing the necessary infrastructure and resources, and to my fellow students for their constructive feedback and suggestions.

Special thanks to the open-source community, particularly the contributors of scikit-learn, NLTK, imbalanced-learn, and Streamlit, whose tools made this project possible.

Finally, I thank my family for their constant support and motivation.

Himanshu Gupta

Abstract

This project presents an end-to-end Natural Language Processing (NLP) system for classifying Amazon product reviews as Positive or Negative. The primary objective is to compare Classical Machine Learning approaches with Deep Learning approaches on real-world, highly imbalanced data.

The dataset consists of 1,597 Amazon product reviews, of which 1,177 contained valid ratings. After preprocessing and removing neutral reviews (ratings of 3), the final dataset contained 1,053 reviews: 977 Positive (rating >= 4) and 76 Negative (rating <= 2), representing a severe class imbalance of approximately 93:7.

Text preprocessing included lowercasing, URL removal, punctuation removal, and stop-word elimination. Features were extracted using TF-IDF (Term Frequency-Inverse Document Frequency) with bigrams. Class imbalance was addressed using SMOTE (Synthetic Minority Oversampling Technique) combined with class_weight='balanced' in Logistic Regression. Hyperparameter tuning was performed using GridSearchCV with 5-fold cross-validation optimizing the macro F1-score.

The final tuned model achieved 92.41% overall accuracy, with Negative class Precision: 33%, Recall: 45%, F1: 38%, and Positive class Precision: 97%, Recall: 95%, F1: 96%. The project also includes a Streamlit web application for real-time sentiment prediction.

Key limitations of TF-IDF — inability to understand word order, context, and sarcasm — motivate the proposed future upgrade to LSTM with Word Embeddings.

Table of Contents

Chapter / Section	Page
1. Introduction	6
2. Literature Review	8
3. System Analysis	9
4. System Design	11
5. Implementation	13
6. Testing	18
7. Results and Discussion	20
8. Conclusion and Future Scope	22
References / Bibliography	23
Appendix	24

Chapter 1 – Introduction

1.1 Project Overview

This project implements an Amazon Review Sentiment Analysis system that automatically classifies product reviews written by customers as either Positive or Negative. The system uses Natural Language Processing (NLP) and Machine Learning (ML) techniques to process raw text and generate predictions.

The project is built to compare Classical ML (TF-IDF + Logistic Regression) against Deep Learning approaches (LSTM) on real-world, highly imbalanced data, providing insights into where each method succeeds and where it fails.

1.2 Problem Statement

Given a product review written by a customer on Amazon, the task is to automatically classify the sentiment of the review as:

- Positive — ratings of 4 or 5 stars
- Negative — ratings of 1 or 2 stars

Manual sentiment analysis is time-consuming and does not scale. With millions of reviews posted on e-commerce platforms daily, there is a critical need for automated systems that can quickly and accurately determine customer sentiment to help businesses understand their customers, identify product issues, and improve services.

1.3 Objectives

1. To preprocess and clean raw Amazon review text data.
2. To handle severe class imbalance using SMOTE and class weighting.
3. To convert text into numerical features using TF-IDF with bigrams.
4. To train and evaluate a Logistic Regression classifier.
5. To perform hyperparameter tuning using GridSearchCV.
6. To deploy the trained model as a real-time Streamlit web application.
7. To identify limitations of the classical ML approach and propose future improvements using LSTM.

1.4 Scope of the Project

The scope of this project includes:

- Binary sentiment classification: Positive vs. Negative (neutral reviews excluded).
- Dataset: Amazon product reviews from publicly available Kaggle dataset.
- NLP techniques: text cleaning, stop-word removal, TF-IDF feature extraction.

- ML technique: Logistic Regression with class balancing.
- Optimization: GridSearchCV with 5-fold cross-validation.
- Web deployment: Streamlit application for real-time prediction.
- Future scope: LSTM with Word Embeddings for improved contextual understanding.

Chapter 2 – Literature Review

2.1 Existing Systems and Related Work

Sentiment analysis has been an active area of research in NLP for over two decades. Early approaches relied on lexicon-based methods using sentiment dictionaries such as SentiWordNet and VADER (Valence Aware Dictionary and sEntiment Reasoner). These rule-based systems assign sentiment scores to individual words and aggregate them to determine overall sentiment.

Bag-of-Words (BoW) and TF-IDF became the standard feature extraction methods for classical ML-based sentiment analysis. Naive Bayes, Support Vector Machines (SVM), and Logistic Regression trained on TF-IDF features demonstrated strong performance on balanced datasets but struggled significantly with imbalanced data — a problem commonly encountered in real-world reviews where positive reviews dominate.

2.2 Challenges in Sentiment Analysis

- Class Imbalance: Real-world review datasets often contain far more positive reviews than negative ones, causing models to be biased toward the majority class.
- Negation Handling: Phrases like 'not bad' are misclassified by word-level models.
- Sarcasm Detection: 'Great, the product broke in one week' — TF-IDF cannot detect irony.
- Mixed Sentiment: Reviews with both positive and negative content in the same sentence.
- Context Dependency: The meaning of words changes based on surrounding context.

2.3 Deep Learning Approaches

Recent advances in deep learning, particularly RNNs (Recurrent Neural Networks), LSTMs (Long Short-Term Memory networks), and Transformer models such as BERT, have significantly improved sentiment analysis performance. These models capture sequential and contextual information that TF-IDF cannot represent.

BERT (Bidirectional Encoder Representations from Transformers) by Google has achieved state-of-the-art results on sentiment analysis benchmarks by leveraging pre-trained language models fine-tuned on domain-specific data.

Chapter 3 – System Analysis

3.1 System Requirements

Hardware Requirements:

- Processor: Intel Core i5 or higher (multi-core recommended for GridSearchCV)
- RAM: Minimum 8 GB (16 GB recommended)
- Storage: 5 GB free space

Software Requirements:

- Operating System: Windows 10 / Linux / macOS
- Programming Language: Python 3.8+
- IDE: Jupyter Notebook / VS Code

3.2 Functional Requirements

8. The system shall accept raw product review text as input.
9. The system shall preprocess input text: lowercase, remove URLs, punctuation, stop-words.
10. The system shall convert text to TF-IDF feature vectors.
11. The system shall classify the review as Positive or Negative.
12. The system shall output the predicted label and confidence probability.
13. The system shall provide a web interface for real-time predictions.

3.3 Non-Functional Requirements

- Performance: Response time for a single prediction should be under 1 second.
- Accuracy: Macro F1-score should exceed 60% on the test set.
- Usability: The Streamlit UI should be simple and intuitive.
- Portability: The system should run on any OS with Python 3.8+.
- Maintainability: Code should be modular with separate preprocessing, training, and inference modules.

3.4 Feasibility Study

Technical Feasibility: Python provides mature libraries (scikit-learn, NLTK, imbalanced-learn, Streamlit) that fully support the implementation of this system. The chosen algorithms are well-documented and computationally feasible on standard hardware.

Economic Feasibility: All libraries and tools used are open-source and free. The dataset is publicly available on Kaggle. Streamlit Community Cloud provides free deployment. Total cost: zero.

Operational Feasibility: The Streamlit web interface makes the system accessible to non-technical users. No domain expertise is required to use the application.

Chapter 4 – System Design

4.1 System Architecture

The system follows a four-stage pipeline architecture:

Stage	Description
1. Data Ingestion	Load raw Amazon review CSV, drop rows with missing ratings
2. Preprocessing	Lowercase, remove URLs/punctuation, remove stop-words, apply sentiment labeling
3. Modeling	TF-IDF vectorization → SMOTE balancing → Logistic Regression with GridSearchCV tuning
4. Deployment	Streamlit web app with real-time prediction pipeline and confidence scores

4.2 Data Flow

Input Flow:

14. Raw review text entered in Streamlit UI
15. clean_text() function: lowercase, remove URLs, punctuation, stop-words
16. gridtfidf.transform() converts cleaned text to TF-IDF vector
17. best_model.predict() outputs Positive (1) or Negative (0)
18. best_model.predict_proba() outputs confidence percentage
19. Result displayed in Streamlit with success/error styling

4.3 Database / Data Schema

Column	Data Type	Description
reviews.text	String	Raw review text from customer
reviews.rating	Float (1.0-5.0)	Star rating given by customer
sentiment	Binary (0/1)	Derived label: 1=Positive, 0=Negative
review_length	Integer	Character count of raw review
Cleaned_Review	String	Preprocessed review text

4.4 Use Case Diagram (Description)

Primary Actors: End User, System (ML Pipeline)

Use Cases:

- UC-1: User enters product review text
- UC-2: System preprocesses the text
- UC-3: System extracts TF-IDF features
- UC-4: System predicts sentiment class
- UC-5: System returns label and confidence score to user

Chapter 5 – Implementation

5.1 Tools and Technologies Used

Tool / Library	Purpose
Python 3.8+	Core programming language
pandas	Data loading, cleaning, and manipulation
numpy	Numerical operations
matplotlib / seaborn	Data visualization and EDA plots
NLTK	Stop-word corpus for text preprocessing
scikit-learn	TF-IDF, Logistic Regression, GridSearchCV, metrics
imbalanced-learn	SMOTE for synthetic oversampling
Streamlit	Web application for real-time prediction
pickle	Model serialization and loading
Jupyter Notebook	Exploratory Data Analysis (EDA)

5.2 Module Description

The project is organized into the following modules:

Module 1: Data Loading and EDA (EDA_AMAZON_REVIEW.ipynb)

- Loads raw Amazon review CSV (1,597 rows, 27 columns)
- Drops rows with missing ratings → 1,177 valid rows
- Applies sentiment labeling: Positive (>= 4), Negative (<= 2), Neutral (== 3, dropped)
- Performs class distribution analysis — identifies 93:7 imbalance
- Analyzes review length distribution and sentiment vs. length relationship

Module 2: Text Preprocessing

The clean_text() function performs the following steps:

20. Convert text to lowercase
21. Remove URLs using regex: re.sub(r'http\S+', '', text)
22. Remove non-alphabetic characters: re.sub(r'[^a-z\s]', '', text)
23. Remove extra whitespace: re.sub(r'\s+', ' ', text).strip()
24. Remove English stop-words using NLTK corpus

Module 3: Feature Engineering (TF-IDF)

TF-IDF (Term Frequency-Inverse Document Frequency) converts cleaned text into a numerical matrix. The final configuration after GridSearchCV tuning:

- max_features=1000: Keep top 1,000 most informative terms
- ngram_range=(1,2): Use unigrams and bigrams
- min_df=3: Ignore terms appearing in fewer than 3 documents
- max_df=0.9: Ignore terms appearing in more than 90% of documents

Module 4: SMOTE Balancing

SMOTE (Synthetic Minority Oversampling Technique) is applied only on the training set to address the 93:7 class imbalance:

- Before SMOTE: Positive=777, Negative=64 in training split
- After SMOTE: Positive=777, Negative=777 (balanced)
- SMOTE creates synthetic negative review vectors interpolated between existing negative samples
- Critically, SMOTE is NOT applied to the test set to preserve real-world evaluation

Module 5: Model Training and Hyperparameter Tuning

GridSearchCV with 5-fold cross-validation was used to find optimal parameters. The parameter grid tested:

- TF-IDF: max_features in [1000, 2000, 3000], ngram_range in [(1,1), (1,2)], min_df in [3, 5], max_df in [0.9, 0.95]
- Logistic Regression: C in [0.01, 0.1, 1, 10, 100], solver in ['liblinear', 'saga'], class_weight=['balanced']
- 240 parameter combinations tested across 5 folds = 1,200 total fits
- Scoring metric: macro F1-score (optimal for imbalanced datasets)
- Best parameters: max_features=1000, ngram_range=(1,2), min_df=3, max_df=0.9, C=1, solver=liblinear, class_weight=balanced

Module 6: Streamlit Web Application (app.py)

The deployment interface provides:

- Text area for pasting any Amazon review
- 'Analyze Sentiment' button triggering the prediction pipeline
- Result displayed with color-coded feedback (green = Positive, red = Negative)
- Confidence percentage displayed for both Positive and Negative classes

5.3 Key Code Explanation

Text Cleaning Function:

```
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'http\S+', '', text)
    text = re.sub(r'[^a-z\s]', '', text)
    words = [w for w in words if w not in stop_words]
    return ' '.join(words)
```

Sentiment Labeling Function:

```
def label_sentiment(rating):
    if rating >= 4:  return 1   # Positive
    elif rating <= 2: return 0   # Negative
    else: return None            # Neutral (dropped)
```

Chapter 6 – Testing

6.1 Testing Methods

- Unit Testing: Individual functions (clean_text, label_sentiment, predict_sentiment) tested with known inputs
- Integration Testing: Full pipeline tested end-to-end from raw text to prediction
- Hold-out Testing: 80/20 train-test split with stratification
- Cross-Validation: 5-fold cross-validation during GridSearchCV hyperparameter tuning

6.2 Test Cases

#	Input Review	Type	Expected	Predicted
1	This product is amazing! Quality is excellent and totally worth the money.	Clear Positive	Positive	Positive (61%)
2	This product is terrible. Very poor quality, not worth the money.	Clear Negative	Negative	Negative (62%)
3	Not bad at all! Initially worried but this product exceeded my expectations.	Hard Positive (negation)	Positive	Negative (54%) FAIL
4	Packaging nice and delivery fast but product is absolutely useless, stopped working in a week.	Mixed Sentiment	Negative	Positive (77%) FAIL

6.3 Test Results

Metric	Negative Class	Positive Class	Overall
Precision	33%	97%	93% (weighted)
Recall	45%	95%	92% (weighted)
F1-Score	38%	96%	93% (weighted)
Accuracy	—	—	92.41%

Chapter 7 – Results and Discussion

7.1 Model Performance Comparison

Metric	Baseline Model	Final Model (GridSearchCV + SMOTE)
Accuracy	94.78%	92.41%
Negative Precision	0% (ignored)	33%
Negative Recall	0% (ignored)	45%
Negative F1	0% (ignored)	38%
Positive F1	97%	96%

7.2 Key Observations

The baseline model (without SMOTE) achieved 94.78% accuracy but was completely blind to negative reviews — always predicting Positive. This is a classic symptom of class imbalance: the model optimizes overall accuracy by ignoring the minority class.

After applying SMOTE and GridSearchCV tuning, overall accuracy dropped slightly to 92.41% but the model now genuinely detects negative reviews (Recall: 45%). This tradeoff — slightly lower overall accuracy in exchange for honest minority class detection — is exactly the desired behavior for imbalanced classification problems.

7.3 Where the Model Fails and Why

Test Case 3 — 'Not bad at all! Initially worried but this product exceeded my expectations.'

- Predicted Negative — INCORRECT. TF-IDF sees 'bad', 'worried' as negative words.
- Root Cause: TF-IDF treats each word independently; 'not bad' = 'bad' in TF-IDF space. No understanding of negation.

Test Case 4 — 'Packaging nice and delivery fast but the product is absolutely useless and stopped working within a week.'

- Predicted Positive — INCORRECT. 'nice', 'fast' dominate TF-IDF scores.
- Root Cause: TF-IDF ignores word position and sentence structure. Positive words at the start outweigh the critical negative conclusion.

7.4 Why LSTM Is the Next Step

TF-IDF Limitation	How LSTM Fixes It
'not bad' → reads 'bad' as negative	Reads full sequence → 'not' flips the meaning
Ignores word position entirely	Processes words left-to-right with memory
Mixed reviews confuse it (positive start wins)	Understands sentiment shifts mid-sentence
Sarcasm detection impossible	Context memory catches subtle tone changes

Chapter 8 – Conclusion and Future Scope

8.1 Summary

This project successfully implemented an end-to-end Amazon Review Sentiment Analysis system using NLP and Machine Learning. The key achievements are:

- Processed and cleaned 1,053 Amazon product reviews with severe class imbalance (93% Positive, 7% Negative).
- Addressed class imbalance using SMOTE + class_weight='balanced', enabling the model to actually detect negative reviews.
- Applied TF-IDF with bigrams as the feature extraction method.
- Optimized the model using GridSearchCV (240 combinations, 5-fold CV, macro F1 scoring).
- Deployed a real-time prediction system using Streamlit.
- Achieved 92.41% overall accuracy with genuine negative class detection (F1: 38%).
- Identified and clearly documented the limitations of the TF-IDF approach.

8.2 Future Improvements

Short-term improvements:

- Collect more negative review data to reduce class imbalance naturally.
- Experiment with n-gram ranges (trigrams), subword tokenization.
- Try other classical ML classifiers: SVM, Random Forest, XGBoost.

Long-term improvements:

- Implement LSTM with Word Embeddings (Word2Vec / GloVe) for sequential context understanding.
- Fine-tune a pre-trained BERT model on the Amazon domain for state-of-the-art performance.
- Extend to multi-class sentiment: Very Positive, Positive, Neutral, Negative, Very Negative.
- Add aspect-based sentiment analysis: identify which product aspects (battery, screen, delivery) are positive/negative.
- Deploy as a REST API using FastAPI for integration with e-commerce platforms.

References / Bibliography

1. Scikit-learn: Machine Learning in Python — Pedregosa et al., JMLR 12, pp. 2825-2830, 2011. [https://scikit-learn.org]

2. NLTK: Natural Language Toolkit — Bird, Steven, Klein, Ewan, and Loper, Edward (2009). Natural Language Processing with Python. O'Reilly Media.

3. SMOTE: Synthetic Minority Over-sampling Technique — Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). Journal of Artificial Intelligence Research, 16, 321-357.

4. TF-IDF — Sparck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. Journal of Documentation, 28(1), 11-21.

5. Logistic Regression — Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer.

6. Streamlit Documentation — https://docs.streamlit.io

7. BERT: Pre-training of Deep Bidirectional Transformers — Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). arXiv:1810.04805.

8. Amazon Review Dataset — Kaggle Public Dataset, accessed 2025.

9. Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets — Lemaitre et al., JMLR 18, 2017.

Appendix

Appendix A: Dataset Statistics

Statistic	Value
Total Raw Records	1,597
Records with Valid Rating	1,177
After Removing Neutral Reviews	1,053
Positive Reviews (Rating >= 4)	977 (92.8%)
Negative Reviews (Rating <= 2)	76 (7.2%)
Mean Review Length (chars)	697
Max Review Length (chars)	19,739
Training Set Size	841 (80%)
Test Set Size	211 (20%)

Appendix B: Project Directory Structure

```
AMAZON_REVIEW_SENTIMENT/
├── data/
│   ├── raw/amazon_review_dataset.csv
│   └── processed/cleaned_data.csv
├── notebooks/
│   └── EDA_AMAZON_REVIEW.ipynb
├── final_models/
│   ├── gridtfidf_vectorizer.pkl
│   └── sentiment_model.pkl
├── app.py (Streamlit Application)
├── requirements.txt
├── README.md
└── .gitignore
```

Appendix C: Requirements

```
ipykernel
numpy
pandas
matplotlib
seaborn
scikit-learn
nltk
regex
joblib
```

```
imblearn
streamlit
```